Лабораторная работа №6. Основы Ansible

Содержание

Содержание

1 Цели лабораторной работы **3**

2 Задачи к лабораторной работе **3**

3 Методические материалы **4**

3.1 Что такое Ansible? 4

3.2 Что такое Ansible Inventories? 5

3.3 Ad-hoc команды 9

3.4 Что такое Ansible Playbooks? 13

3.5 Структура каталогов 27

4 Ход работы **29**

4.1 Подготовка окружения 29

4.2 Создание инвентаря 35

4.3 Создание переменных группы 36

4.4 Проверка доступности хостов 36

4.5 Создание плейбука 36

4.6 Запуск плейбука и проверка выполнения 37

5 Контрольные вопросы **38**

1 Цели лабораторной работы

- Познакомиться с инструментом Ansible;
- Изучить формат описания конфигураций Ansible.

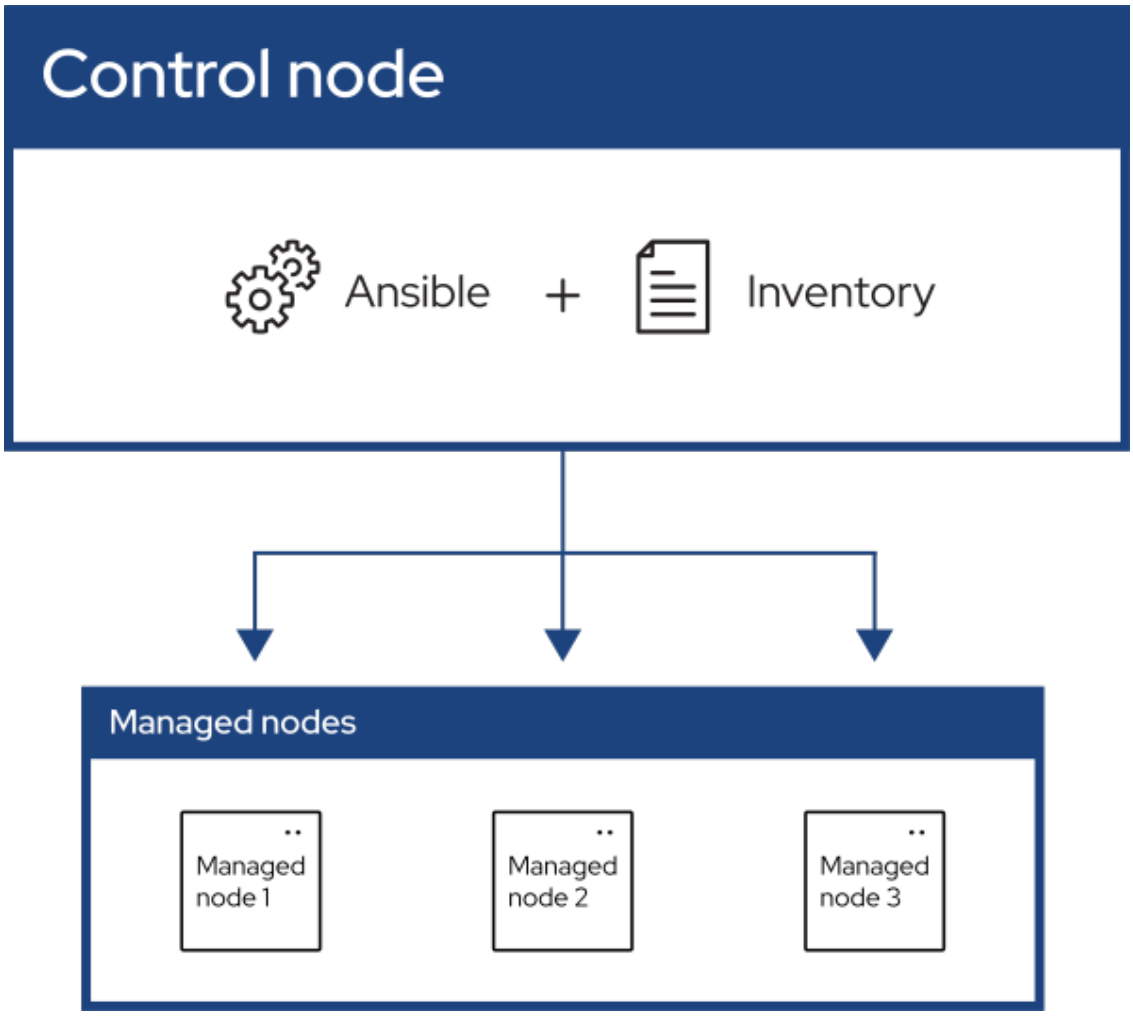
2 Задачи к лабораторной работе

1. Изучить методические материалы к лабораторной работе;
2. Подготовить окружение;
3. Описать файл инвентаря с именем *hosts*, создав группы:
   - *staging_serves*, состоящий из двух хостов;
   - *prod_serves*, состоящий из одного хоста;
   - *all_serves*, объединяющий две предыдущие группы.
4. Описать переменные для группы *all_serves*;
5. Проверить доступность серверов через ad-hoc команду;
6. Написать Playbook для установки и запуска nginx.

3 Методические материалы

3.1 Что такое Ansible?

Ansible - это система управления конфигурациями. Ansible автоматизирует управление удаленными системами и контролирует их желаемое состояние.



Задачи, которые выполняет Ansible:

- Устраняет повторы и упрощает рабочие процессы;
- Управляет конфигурациями системы и поддерживает их;
- Продолжительное развертывание сложного программного обеспечения;
- Выполняет rolling updates («плавающие обновления») с нулевым временем простоя системы.

Ansible основан на следующих принципах:

- **Agent-less архитектура**. Не требует установки агентского ПО на управляемые узлы, что снижает эксплуатационные расходы;
- **Простота**. Плейбуки описываются на языке YAML, также Ansible децентрализован: для доступа к удаленным машинам используется SSH;
- **Масштабируемость и гибкость**. Легкая и быстрая масштабируемость системы благодаря модульной конструкции;
- **Идемпотентность и предсказуемость**. Идемпотентность - свойство объекта или операции при повторном применении операции с одними и теми же параметрами давать один и тот же результат.

3.2 Что такое Ansible Inventories?

Инвентари Ansible - это файлы, которые содержат в себе описание всех узлов, которыми будет управлять Ansible. Инвентари также помогают логически организовывать хосты в группы. Ansible поддерживает несколько способов описания файла инвентаря (yaml, ini и т.д.). Для добавления узла в инвентарь в самом простом случае достаточно указать его IP-адрес или доменнное имя.

```
mail.example.com

[webservers]
foo.example.com
bar.example.com

[dbservers]
one.example.com
two.example.com
three.example.com
```

По умолчанию Ansible неявно создаёт две группы: `all` - в неё входят все хосты; `ungrouped` - в неё входят все хосты, которые не состоят в определённых пользователем группах. Следовательно, каждый хост будет находиться минимум в двух группах.

В примере выше явно заданы две группы: `webservers` и `dbservers`. Хост `mail.example.com` относится к `ungrouped`.

Каждый узел может входить в несколько групп одновременно. Вы можете группировать, используя вопросы:

- Что? - приложение, стек, сервис (к примеру: СУБД, веб-серверы, API и т.д.);
- Где? - датацентры, географическое расположение (к примеру: запад, восток);
- Когда? - различные этапы разработки (dev, stagging, prod)

```
[webservers]
foo.example.com
bar.example.com

[dbservers]
one.example.com
two.example.com
three.example.com

[west]
foo.example.com
bar.example.com
one.example.com

[east]
one.example.com
two.example.com

[prod]
bar.example.com
one.example.com

[dev]
foo.example.com
two.example.com

[stagging]
three.example.com
```

Также между группами может быть установлена связь родитель-потомок. Для создания такой связи используется:

- суффикс `:children` в `INI` формате;
- запись `children:` в `YAML` формате;

```
[albania]
tirana.example.al

[montenegro]
podgorica.example.me

[greece]
athens.example.gr

[norway]
oslo.example.no

[sweden]
stockholm.example.se

[balkans:children]
albania
montenegro
greece

[scandinavia:children]
norway
sweden

[europe:children]
balkans
scandinavia
```

В данном примере хосты объединены по группам стран, а те в свою очередь - по географическому признаку. Родительскими группами являются `balkans`, `scandinavia` и `europe`.

У дочерних групп есть несколько свойств:

- Любой хост, входящий в дочернюю группу, автоматически входит в родительскую группу.
- Группы могут иметь несколько родителей и потомков, но не циклические отношения.
- Хосты также могут находиться в нескольких группах, но во время выполнения будет существовать только один экземпляр хоста. Ansible объединяет данные из нескольких групп.

Существует возможность задавать переменные на уровне хоста и группы:

```
[albania]
tirana.example.al http_port=80 maxRequestsPerChild=808

[montenegro]
podgorica.example.me

[greece]
athens.example.gr

[balkans:children]
albania
montenegro
greece

[balkans:vars]
ansible_user=someuser
ansible_password=1234
```

В примере выше `http_port=80` и `maxRequestsPerChild=808` переменные хоста `tirana.example.al`. Переменные группы записываются в отдельном блоке с суффиксом `:vars`. Переменные `ansible_user` и `ansible_password=1234` будут относиться ко всем хостам группы `balkans`.

По умолчанию переменные объединяются/расплющиваются для конкретного хоста перед запуском сценария. Это позволяет Ansible сосредоточиться на хосте и задаче, поэтому группы не выживают вне инвентаря и соответствия хосту. По умолчанию Ansible перезаписывает переменные, включая те, которые определены для группы и/или хоста. Справедлив следующий приоритет (от наименьшего к наибольшему):

- все группы (потому что она является "родителем"всех остальных групп);
- родительская группа;
- дочерняя группа;
- хост.

Для лучшей организации переменных Ansible предоставляет возможность хранения переменных в каталогах `group_vars` для переменных групп и `host_vars` для переменных хоста. Внутри директории `group_vars` должны находиться yaml-файлы, имена которых совпадают с именем группы, к которым относятся переменные.

Предыдущий пример можно переписать так:

```
[albania]
tirana.example.al

[montenegro]
podgorica.example.me

[greece]
athens.example.gr

[balkans:children]
albania
montenegro
greece
```

./ansible/hosts

```
ansible_user: someuser
ansible_password: 1234
```

./ansible/group_vars/balkans

3.3 Ad-hoc команды

Ad-hoc команда - команда (модуль) Ansible, использующаяся для выполнения одной задачи на управляемых узлах. Команды ad hoc - это быстро и просто, но они не могут быть использованы повторно.

Хотя Ansible предназначен для автоматизации задач с высоким уровнем переиспользования, ad-hoc команды полезны в случаях редко выполняемых действий (к примеру, выключение всех машин).

Структура команды выглядит следующим образом:

```
$ ansible [pattern] -m [module] -a "[module options]"
```

Аргумент `[pattern]` служит для определения хоста/группы хостов, на которых будет выполнена команда. В самом простом случае является названием группы из файла инвентаря или `all` для указания всех хостов.

Аргумент `[module]` задает команду, которая будет исполнятся на всех хостах.

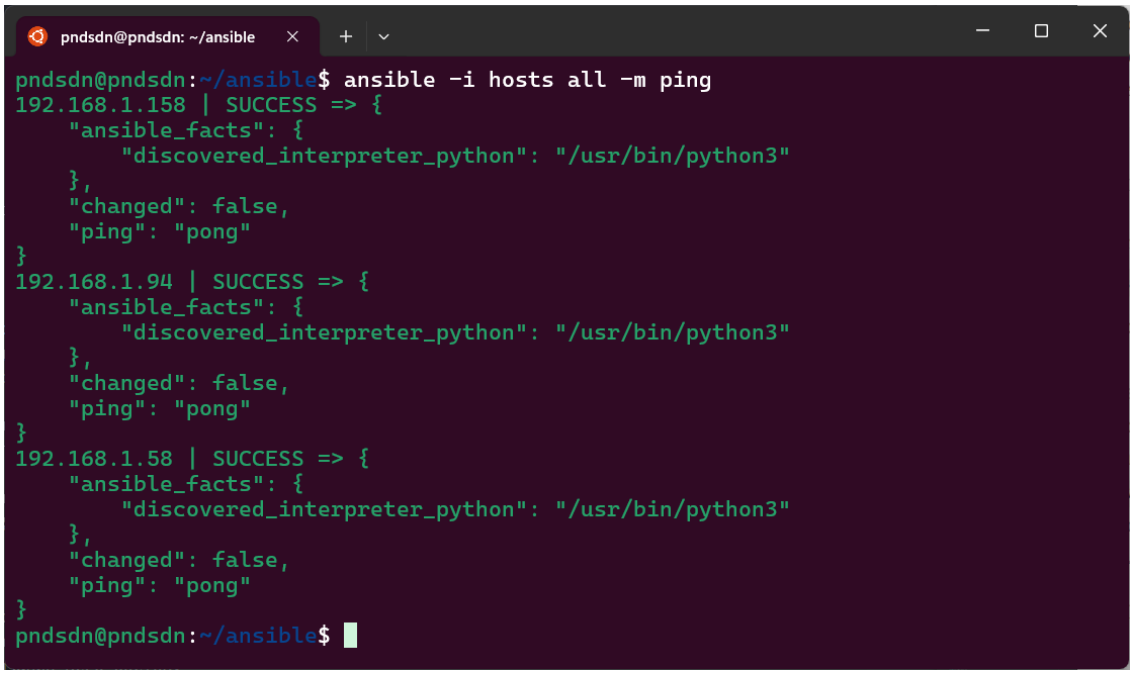
Опциональный флаг `-a` принимает параметры для модуля либо через синтаксис `key-value`, либо в виде JSON-строки, начинающейся с `{` и заканчивающейся `}` для более сложной структуры опций.

Рассмотрим пару примеров:

1. Проверить доступность всех серверов

   ```
   $ ansible -i hosts all -m ping
   ```

   `-i <file>` указывает на файл инвентаря



2. Вывести содержимое домашней директории для группы хостов `staging_servers`

   ```
   $ ansible -i hosts staging_servers -m shell -a 'ls -lA $HOME'
   ```

```
pndsdn@pndsdn: ~/ansible
pndsdn@pndsdn:~/ansible$ ansible -i hosts staging_servers -m shell -a 'ls -lA $HOME'
192.168.1.94 | CHANGED | rc=0 >>
total 28
drwx------ 3 sopov-a-a sopov-a-a 4096 Jan 12 13:24 .ansible
-rw------- 1 sopov-a-a sopov-a-a  202 Jan 12 16:08 .bash_history
-rw-r--r-- 1 sopov-a-a sopov-a-a  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 sopov-a-a sopov-a-a 3771 Mar 31  2024 .bashrc
drwx------ 2 sopov-a-a sopov-a-a 4096 Jan 10 19:04 .cache
-rw-r--r-- 1 sopov-a-a sopov-a-a  807 Mar 31  2024 .profile
drwx------ 2 sopov-a-a sopov-a-a 4096 Jan 10 19:04 .ssh
-rw-r--r-- 1 sopov-a-a sopov-a-a    0 Jan 12 13:15 .sudo_as_admin_successful
192.168.1.158 | CHANGED | rc=0 >>
total 40
drwx------ 3 sopov-a-a sopov-a-a 4096 Jan 12 13:14 .ansible
-rw------- 1 sopov-a-a sopov-a-a  948 Jan 12 16:08 .bash_history
-rw-r--r-- 1 sopov-a-a sopov-a-a  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 sopov-a-a sopov-a-a 3771 Mar 31  2024 .bashrc
drwx------ 2 sopov-a-a sopov-a-a 4096 Jan 10 18:57 .cache
-rw------- 1 sopov-a-a sopov-a-a   20 Jan 11 13:59 .lesshst
-rw-r--r-- 1 sopov-a-a sopov-a-a  807 Mar 31  2024 .profile
drwx------ 2 sopov-a-a sopov-a-a 4096 Jan 11 14:27 .ssh
-rw-r--r-- 1 sopov-a-a sopov-a-a    0 Jan 10 19:07 .sudo_as_admin_successful
-rw------- 1 sopov-a-a sopov-a-a 4565 Jan 11 14:27 .viminfo
pndsdn@pndsdn:~/ansible$
```

3. Скопировать локальный файл `somefile` в домашнюю директорию для:

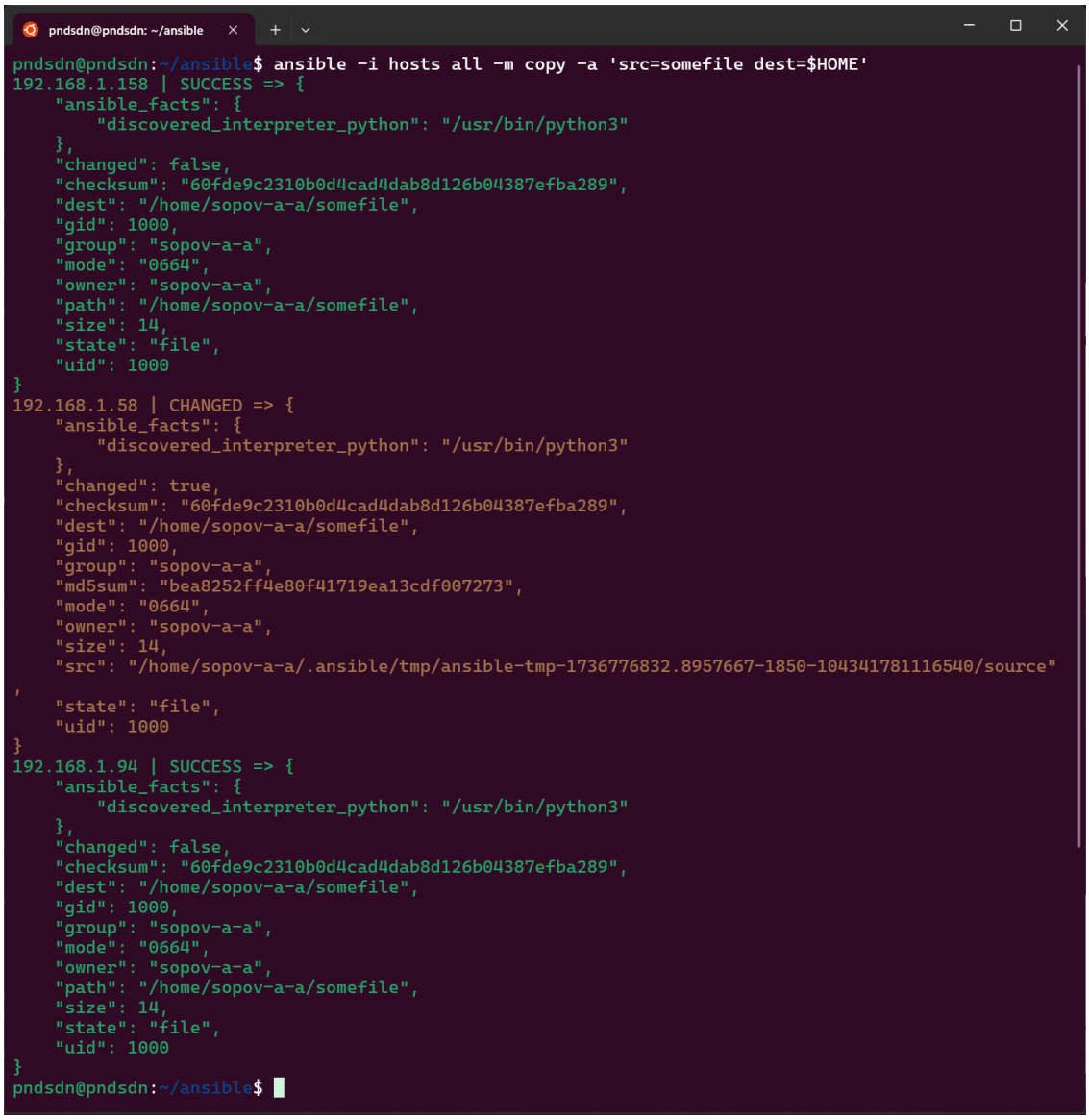
   3.1 группы `staging_servers`

```
$ ansible -i hosts staging_servers -m copy -a 'src=somefile dest=
  $HOME'
```

```
pndsdn@pndsdn: ~/ansible
pndsdn@pndsdn:~/ansible$ ansible -i hosts  staging_servers -m copy -a 'src=somefile dest=$HOME'
192.168.1.94 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": true,
    "checksum": "60fde9c2310b0d4cad4dab8d126b04387efba289",
    "dest": "/home/sopov-a-a/somefile",
    "gid": 1000,
    "group": "sopov-a-a",
    "md5sum": "bea8252ff4e80f41719ea13cdf007273",
    "mode": "0664",
    "owner": "sopov-a-a",
    "size": 14,
    "src": "/home/sopov-a-a/.ansible/tmp/ansible-tmp-1736776728.5886161-1751-65602362181421/source",
    "state": "file",
    "uid": 1000
}
192.168.1.158 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": true,
    "checksum": "60fde9c2310b0d4cad4dab8d126b04387efba289",
    "dest": "/home/sopov-a-a/somefile",
    "gid": 1000,
    "group": "sopov-a-a",
    "md5sum": "bea8252ff4e80f41719ea13cdf007273",
    "mode": "0664",
    "owner": "sopov-a-a",
    "size": 14,
    "src": "/home/sopov-a-a/.ansible/tmp/ansible-tmp-1736776728.4985142-1750-24596719013752/source",
    "state": "file",
    "uid": 1000
}
pndsdn@pndsdn:~/ansible$
```

3.2 всех хостов

```
$ ansible -i hosts all -m copy -a 'src=somefile dest=$HOME'
```



Обратите внимание на вывод команд. В пункте 3.1 для хостов `192.168.1.94` и `192.168.1.158` статус изменения установлен в истину (`"changed": true`).

В пукте 3.2 эти же хосты имеют статус `"changed": false`, но задача всё равно завершилась успешно, так как этот файл уже существовал по заданному пути и не было необходимости его заново копировать.

3.4 Что такое Ansible Playbooks?

Плейбуки Ansible - это повторяющаяся, многократно используемая, простая система управления конфигурацией и развертывания на нескольких машинах, которая хорошо подходит для развертывания сложных приложений. Если вам нужно выполнить задачу с помощью Ansible более одного раза, напишите плейбуки и поместите его под контроль исходного кода. Затем вы сможете использовать плейбук для распространения новой конфигурации или подтверждения конфигурации удаленных систем. Проще говоря, это файл, который содержит список команд и условия их выполнения для конфигурирования хостов.

Плейбуки могут:

- объявлять конфигурации;
- организовывать этапы любого процесса, выполняемого вручную, на нескольких наборах машин в определенном порядке;
- запускать задачи синхронно или асинхронно.

Плейбуки описываются в формате YAML, состоят из набора Пьес (Plays). Каждая пьеса выполняет часть общей цели плейбука, запуская одну или несколько задач. Каждая задача вызывает модуль Ansible.

Выполнение плейбука. Пьесы выполняются в порядке «сверху-вниз». Задачи внутри каждой пьесы выполняются в таком же порядке. Как минимум, каждая пьеса определяет две вещи:

- управляемые узлы, используя шаблон;
- по крайней мере одно задание для выполнения.

Ниже представлен пример плейбука с двумя пьесами: первая обновляет веб-сервер Apache до последней версии и записывает файл конфигурации; вторая обновляет СУБД PostreSQL до последней версии и проверяет, что сервис запущен.

```
---
- name: Update web servers  # имя пьесы
  hosts: webservers         # указывает на группу хостов
```

```
  remote_user: root          # имя пользователя для ssh соединения

  tasks:                     # список задач
  - name: Ensure apache is at the latest version
    ansible.builtin.yum:     # использовать модуль пакетного менеджера yum
      name: httpd            # имя пакета
      state: latest          # желаемое состояние

  - name: Write the apache config file
    ansible.builtin.template:
      src: /srv/httpd.j2
      dest: /etc/httpd.conf

- name: Update db servers
  hosts: databases
  remote_user: root

  tasks:
  - name: Ensure postgresql is at the latest version
    ansible.builtin.yum:
      name: postgresql
      state: latest

  - name: Ensure that postgresql is started
    ansible.builtin.service:
      name: postgresql
      state: started
```

Выполнение задач. По умолчанию Ansible выполняет каждую задачу по порядку, по очереди, на всех машинах, соответствующих шаблону хоста. Каждая задача выполняет модуль с определенными аргументами. Когда задача выполнена на всех целевых машинах, Ansible переходит к следующей задаче. Можно использовать стратегии, чтобы изменить это поведение по умолчанию. В рамках каждой пьесы Ansible применяет одни и те же директивы задач ко всем хостам. Если на каком-то узле задача не сработала, Ansible исключает этот узел из ротации до конца плейбука.

Когда вы запускаете плейбук, Ansible возвращает информацию о соединениях, строках имен всех ваших пьес и задач, о том, удалось или не удалось выполнить

каждую задачу на каждой машине, и о том, были ли внесены изменения в каждую задачу на каждой машине. В конце выполнения плейбука Ansible предоставляет сводную информацию об узлах, на которые были направлены действия, и о том, как они были выполнены.

Условия. Задачи можно запускать по определённым условиям. К примеру, можно запускать различные задачи для установки пакетов в дистрибутивах с разными пакетными менеджерами.

```
---
- name: Install Apache
  hosts: all

  tasks:
  - name: Install Apache Web Server for RedHat
    yum: name=httpd state=latest
    when: ansible_os_family == "RedHat"

  - name: Install Apache Web Server for Debian
    apt: name=apache2 state=latest
    when: ansible_os_family == "Debian"
```

Инструкция `when` задаёт условие, где в качестве тела проверяется является ли семейство ОС «RedHat» или «Debian».

Циклы. Ansible предлагает ключевые слова `loop`, `with_<lookup>` и `until` для многократного выполнения задачи. Примеры часто используемых циклов включают изменение прав собственности на несколько файлов и/или каталогов с помощью модуля file, создание нескольких пользователей с помощью модуля user и повторение шага опроса до достижения определенного результата.

- `loop` появился с версии Ansible 2.5, как простой способ создания циклов и рекомендуется для большенства сценариев использования;
- `with_<lookup>` не считается устаревшим и останется актуальным в обозримом будущем;
- `loop` и `with_<lookup>` являются взаимоисключающими. Хотя их можно вложить в цикл until, это повлияет на каждую итерацию цикла.

Сравнение циклов.

- Обычный случай использования `until` связан с задачами, которые, скорее всего, не будут выполнены, а `loop` и `with_<lookup>` предназначены для повторяющихся задач с небольшими вариациями;
- `loop` и `with_<lookup>` будут запускать задачу один раз для каждого элемента списка, используемого в качестве входных данных, а `until` будет повторять задачу до тех пор, пока не будет выполнено условие;
- Ключевое слово `loop` эквивалентно `with_list` и является лучшим выбором для простых циклов;
- Ключевое слово `until` принимает "конечное условие"(выражение, возвращающее `True` или `False`), которое "неявно шаблонизировано"(нет необходимости в {{ }}), обычно основанное на переменной, которую вы зарегистрировали для задачи;
- `loop_control` влияет на `loop` и `with_<lookup>`, но не на `until`, который имеет свои собственные ключевые слова-компаньоны: `retries` и `delay`.

Использование циклов. Повторяющиеся задачи могут быть написаны как стандартные циклы над простым списком строк. Список можно задать непосредственно в задаче.

```
- name: Add several users
  ansible.builtin.user:
    name: "{{ item }}"
    state: present
    groups: "wheel"
  loop:
     - testuser1
     - testuser2
```

Вы можете определить список в файле переменных или в разделе 'vars' вашей пьесы, а затем ссылаться на имя списка в задании.

```
loop: "{{ somelist }}"
```

Примеры выше являются эквивалентами для

```
- name: Add user testuser1
  ansible.builtin.user:
    name: "testuser1"
    state: present
    groups: "wheel"

- name: Add user testuser2
  ansible.builtin.user:
    name: "testuser2"
    state: present
    groups: "wheel"
```

Для некоторых плагинов можно передавать список непосредственно в параметр. Большинство модулей упаковки, таких как `yum` и `apt`, имеют такую возможность. При наличии такой возможности передача списка в качестве параметра лучше, чем циклическое выполнение задачи. Например

```
- name: Optimal yum
  ansible.builtin.yum:
    name: "{{ list_of_packages }}"
    state: present

- name: Non-optimal yum, slower and may cause issues with
   interdependencies
  ansible.builtin.yum:
    name: "{{ item }}"
    state: present
  loop: "{{ list_of_packages }}"
```

Блоки. Блоки создают логические группы задач. Блоки также предлагают способы обработки ошибок в задачах, аналогичные обработке исключений во многих языках программирования.

```
tasks:
- name: Install, configure, and start Apache
    when: ansible_facts['distribution'] == 'CentOS'
    block:
    - name: Install httpd and memcached
        ansible.builtin.yum:
        name:
```

```
        - httpd
        - memcached
        state: present

    - name: Apply the foo config template
        ansible.builtin.template:
        src: templates/src.j2
        dest: /etc/foo.conf

    - name: Start service bar and enable it
        ansible.builtin.service:
        name: bar
        state: started
        enabled: True
    become: true
    become_user: root
    ignore_errors: true
```

В приведенном примере условие `when` будет оцениваться перед выполнением Ansible каждой из трех задач в блоке. Все три задачи также наследуют директивы повышения привилегий, запускаясь от имени пользователя `root`. Наконец, параметр `ignore_errors: true` гарантирует, что Ansible продолжит выполнение плейбука, даже если некоторые из задач не будут выполнены.

Обработчики. Иногда требуется, чтобы задача запускалась только при изменении машины. Например, вы можете захотеть перезапустить службу, если задача обновляет конфигурацию этой службы, но не перезапускать ее, если конфигурация не изменилась. Для решения этой задачи Ansible использует обработчики. Обработчики - это задачи, которые запускаются только при получении уведомления.

```
---
- name: Verify apache installation
    hosts: webservers
    vars:
    http_port: 80
    max_clients: 200
    remote_user: root
    tasks:
    - name: Ensure apache is at the latest version
```

```
        ansible.builtin.yum:
        name: httpd
        state: latest

    - name: Write the apache config file
        ansible.builtin.template:
        src: /srv/httpd.j2
        dest: /etc/httpd.conf
        notify:
        - Restart apache

    - name: Ensure apache is running
        ansible.builtin.service:
        name: httpd
        state: started

    handlers:
    - name: Restart apache
        ansible.builtin.service:
        name: httpd
        state: restarted
```

В этом примере плейбука сервер Apache перезапускается обработчиком после выполнения всех задач плейбука.

Уведомление обработчиков. Задачи могут указывать одному или нескольким обработчикам на выполнение с помощью ключевого слова `notify`, которое может быть применено к задаче и принимает список имен обработчиков, которые будут уведомлены об изменении задачи. В качестве альтернативы можно указать строку, содержащую одно имя обработчика. Следующий пример демонстрирует, как несколько обработчиков могут быть уведомлены одной задачей:

```
tasks:
- name: Template configuration file
    ansible.builtin.template:
    src: template.j2
    dest: /etc/foo.conf
    notify:
    - Restart apache
```

```
    - Restart memcached

handlers:
    - name: Restart memcached
    ansible.builtin.service:
        name: memcached
        state: restarted

    - name: Restart apache
    ansible.builtin.service:
        name: apache
        state: restarted
```

В приведенном выше примере обработчики выполняются при смене задачи в следующем порядке: `Restart memcached`, `Restart apache`. Обработчики выполняются в том порядке, в котором они определены в разделе `handlers`, а не в порядке, указанном в операторе `notify`. Если уведомить один и тот же обработчик несколько раз, он будет выполнен только один раз, независимо от того, сколько задач его уведомили. Например, если несколько задач обновляют конфигурационный файл и уведомляют обработчик о необходимости перезапустить Apache, Ansible выполняет Apache только один раз, чтобы избежать ненужных перезапусков.

Уведомление и циклы. Задачи могут использовать циклы для уведомления обработчиков. Это особенно полезно в сочетании с переменными для запуска нескольких динамических уведомлений. Обратите внимание, что обработчики запускаются при изменении задачи в целом. При использовании цикла измененное состояние устанавливается при изменении любого из элементов цикла. То есть при любом изменении срабатывают все обработчики.

```
tasks:
- name: Template services
  ansible.builtin.template:
    src: "{{ item }}.j2"
    dest: /etc/systemd/system/{{ item }}.service
  # Note: if *any* loop iteration triggers a change, *all* handlers are
    run
  notify: Restart {{ item }}
  loop:
```

```
    - memcached
    - apache

handlers:
  - name: Restart memcached
    ansible.builtin.service:
      name: memcached
      state: restarted

  - name: Restart apache
    ansible.builtin.service:
      name: apache
      state: restarted
```

В приведенном выше примере и `memcached`, и `apache` будут перезапущены при изменении одного из файлов шаблона, но ни один из них не будет перезапущен, если ни один из файлов не изменится.

Именование обработчиков. Обработчики должны быть названы, чтобы задачи могли уведомлять их с помощью ключевого слова `notify` (значение в `notify` должно полностью совпадать с названием обработчика). В качестве альтернативы обработчики могут использовать ключевое слово `listen`. Используя это ключевое слово, обработчики могут прослушивать темы, которые могут группировать несколько обработчиков следующим образом:

```
tasks:
  - name: Restart everything
    command: echo "this task will restart the web services"
    notify: "restart web services"

handlers:
  - name: Restart memcached
    service:
      name: memcached
      state: restarted
    listen: "restart web services"

  - name: Restart apache
    service:
```

```
      name: apache
      state: restarted
    listen: "restart web services"
```

Уведомление темы `restart web services` приводит к выполнению всех обработчиков, прослушивающих эту тему, независимо от того, как эти обработчики названы.

Такое использование значительно упрощает запуск нескольких обработчиков. Кроме того, он отделяет обработчики от их имен, что упрощает совместное использование обработчиков в плейбуках и ролях (особенно при использовании сторонних ролей из общего источника, такого как Ansible Galaxy).

Каждый обработчик должен иметь глобально уникальное имя. Если определено несколько обработчиков с одним и тем же именем, только последний из них, загруженный в игру, может быть уведомлен и выполнен, эффективно затеняя все предыдущие обработчики с тем же именем.

Существует только одна глобальная область действия для обработчиков (имена обработчиков и темы прослушивания), независимо от того, где определены обработчики. Это также относится к обработчикам, определенным в ролях

Роли. Роли позволяют автоматически загружать связанные переменные, файлы, задачи, обработчики и другие артефакты Ansible на основе известной структуры файлов. Сгруппировав содержимое по ролям, вы сможете легко использовать их повторно и делиться ими с другими пользователями.

Структура каталогов. Роль Ansible имеет определенную структуру каталогов с семью основными стандартными каталогами (По умолчанию Ansible будет искать в большинстве каталогов ролей файл `main.yml` для соответствующего содержимого, а также `main.yaml` и `main`):

- `tasks/main.yaml` - список задач, которые роль предоставляет пьесе для выполнения;
- `handlers/main.yaml` - обработчики, которые импортируются в родительскую пьесу для использования этой ролью или другими ролями и задачами в пьесе;
- `defaults/main.yaml` - очень низкий приоритет значений для переменных, предоставляемых ролью (см. раздел "Использование переменных"для получения до-

полнительной информации). Собственные значения по умолчанию роли будут иметь приоритет над значениями по умолчанию других ролей, но любые/все другие источники переменных будут переопределять эти переменные;

- `vars/main.yaml` - переменные с высоким приоритетом, предоставляемые ролью для пьесы;
- `files/stuff.txt` - один или несколько файлов, доступных для роли и ее дочерних элементов;
- `templates/something.j2` - шаблоны для использования в роли или дочерних ролях;
- `meta/main.yaml` - метаданные для роли, включая её зависимости и необязательные метаданные Galaxy, такие как поддерживаемые платформы. Это необходимо для загрузки в Galaxy в качестве отдельной роли;

Хотя бы один из каталогов должен быть включён в каждую роль. Вы можете опустить все каталоги, которые роль не использует.

Вы можете добавить другие YAML-файлы в некоторые директории, но они не будут использоваться по умолчанию. Их можно включать/импортировать напрямую или указывать при использовании `include_role` / `import_role`. Например, вы можете поместить специфические для платформы задачи в отдельные файлы и ссылаться на них в файле `tasks/main.yml`:

```
# roles/example/tasks/main.yml
- name: Install the correct web server for RHEL
    import_tasks: redhat.yml
    when: ansible_facts['os_family']|lower == 'redhat'

- name: Install the correct web server for Debian
    import_tasks: debian.yml
    when: ansible_facts['os_family']|lower == 'debian'

# roles/example/tasks/redhat.yml
- name: Install web server
    ansible.builtin.yum:
    name: "httpd"
```

```
    state: present

# roles/example/tasks/debian.yml
- name: Install web server
    ansible.builtin.apt:
    name: "apache2"
    state: present
```

Или вызывайте эти задачи непосредственно при загрузке роли, что позволяет обойтись без файла `main.yml`:

```
- name: include apt tasks
  include_role:
      name: package_manager_bootstrap
      tasks_from: apt.yml
  when: ansible_facts['os_family'] == 'Debian'
```

Хранение и поиск ролей. По умолчанию Ansible ищет роли в следующих местах:

- в коллекциях, если вы их используете;
- в директории `roles/`, относительно файла плейбука;
- в настроенном пути `roles_path`. По умолчанию путь поиска `/.ansible/roles:/usr/share/ansible/roles:/etc/ansible/roles`;
- в директории, где находится файл плейбука.

Использование ролей. Роли можно использовать следующим образом:

- на уровне пьесы с помощью опции `roles`: Это классический способ использования ролей в пьесе;
- на уровне задач с помощью `include_role`: Вы можете динамически повторно использовать роли в любом месте раздела `tasks` пьесы с помощью `include_role`;
- на уровне задач с помощью `import_role`: Вы можете статически повторно использовать роли в любом месте раздела `tasks` пьесы с помощью `import_role`;
- как зависимость от другой роли.

Использование ролей на уровне пьесы. Классический (оригинальный) способ использования ролей - это опция `roles` для данной пьесы:

```
---
  - hosts: webservers
    roles:
      - common
      - webservers
```

Когда вы используете опцию `roles` на уровне пьесы, каждая роль 'x' ищет `main.yml` (также `main.yaml` и `main`) в следующих каталогах:

- `roles/x/tasks/`;
- `roles/x/handlers/`;
- `roles/x/vars/`;
- `roles/x/defaults/`;
- `roles/x/meta/`;
- Любые задачи copy, script, template или include (в роли) могут ссылаться на файлы в roles/x/files,templates,tasks/ (dir зависит от задачи) без необходимости указывать путь к ним относительно или абсолютно.

Когда вы используете опцию `roles` на уровне пьесы, Ansible рассматривает роли как статический импорт и обрабатывает их во время парсинга плейбука. Ansible выполняет каждое действие в таком порядке:

1. Любые `pre_tasks`, определенные в пьесе;
2. Любые обработчики, запускаемые `pre_tasks`;
3. Каждая роль, перечисленная в `roles:`, в указанном порядке. Любые зависимости роли, определенные в `meta/main.yml` роли, запускаются первыми, с учетом фильтрации тегов и условий;
4. Любые `tasks`, определенные в пьесе;
5. Любые обработчики, запускаемые ролями или задачами;

6. Любые `post_tasks`, определенные в пьесе;

7. Любые обработчики, запускаемые `post_tasks`.

Including roles: динамическое переиспользование. С помощью `include_role` можно динамически повторно использовать роли в любом месте раздела `tasks` пьесы. В то время как роли, добавленные в разделе `include_roles`, запускаются перед любыми другими задачами в пьесе, включенные роли запускаются в том порядке, в котором они определены. Если перед задачей `include_roles` есть другие задачи, они будут выполняться первыми.

```
---
- hosts: webservers
  tasks:
    - name: Print a message
      ansible.builtin.debug:
        msg: "this task runs before the example role"

    - name: Include the example role
      include_role:
        name: example

    - name: Print a message
      ansible.builtin.debug:
        msg: "this task runs after the example role"
```

Importing roles: статическое переиспользование. Вы можете повторно использовать роли статически в любом месте раздела `tasks` пьесы с помощью `import_role`. Поведение такое же, как и при использовании ключевого слова `roles`. Например:

```
---
- hosts: webservers
  tasks:
    - name: Print a message
      ansible.builtin.debug:
        msg: "before we run our role"

    - name: Import the example role
```

```
      import_role:
        name: example

    - name: Print a message
      ansible.builtin.debug:
        msg: "after we ran our role"
```

3.5 Структура каталогов

В Ansible хорошей практикой считается поддержание определённой файловой структуры, которая упрощает понимание и навигацию по модулям, а также взаимодействие с ними.

Ниже приведён пример одного из возможных вариантов организации файлов.

```
ansible.cfg              # файл конфигурации

production               # файл инвентаря для окружения 'production'
staging                  # файл инвентаря для окружения 'staging'

group_vars/              # каталог с файлами переменных определённых групп
    group1.yml
    group2.yml
host_vars/               # каталог с файлами переменных определённых хостов
    hostname1.yml
    hostname2.yml

site.yml                 # главный playbook
webservers.yml           # playbook для уровня 'webservers'
dbservers.yml            # playbook для уровня 'dbservers'

roles/                   # каталог ролей
    common/              # роль 'common'
        tasks/
            main.yml
        handlers/
            main.yml
        templates/
            ntp.conf.j2
        vars/
```

```
            main.yml
        defaults/
            main.yml
        meta/
            main.yml

    webtier/            # такая же структура, как и 'common'
    monitoring/
    fooapp/
```

4 Ход работы

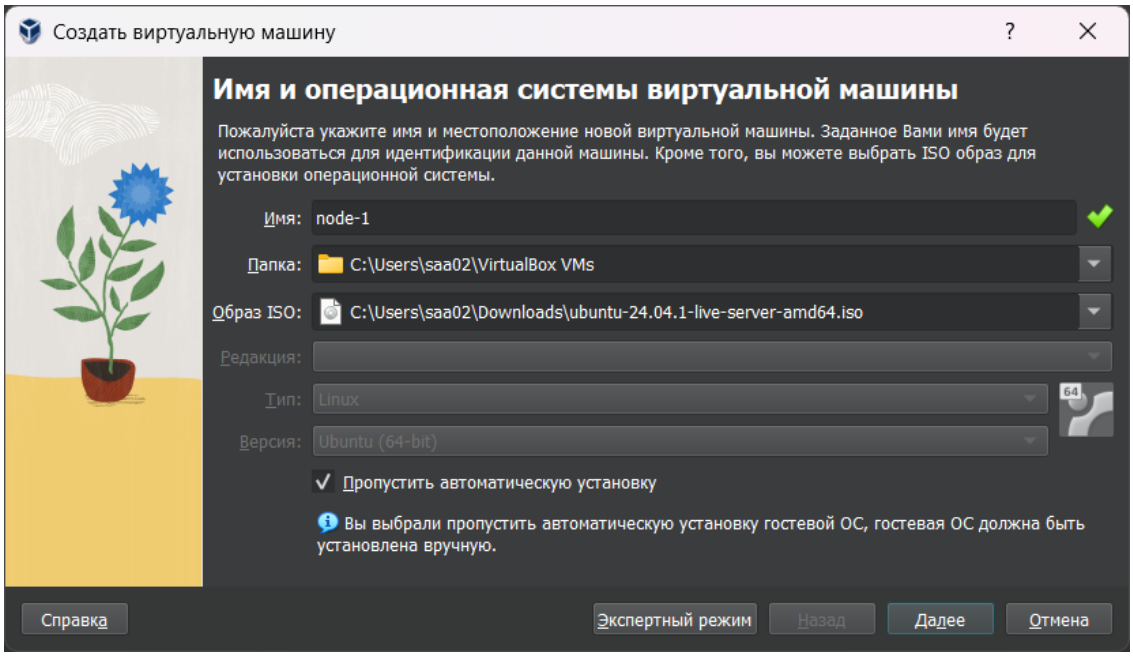
4.1 Подготовка окружения

Наше окружение будет состоять из трёх управляемых машин. Управлять же мы будет с хоста. Ansible работает только на UNIX-подобных системах, поэтому если ваша ОС - Windows, то у вас должен быть установлен WSL2.

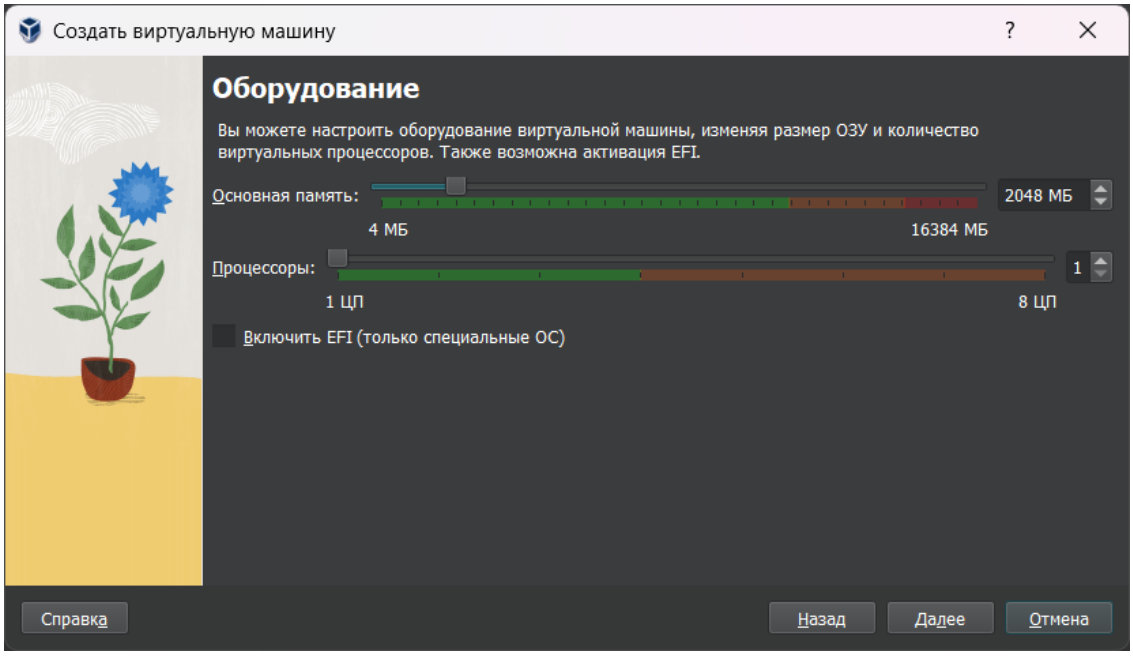
Для запуска ВМ будем использовать Oracle VirtualBox. Скачайте и установите VirtualBox. Скачайте Ubuntu Server.

Запустите VirtualBox и нажмите «Создать».

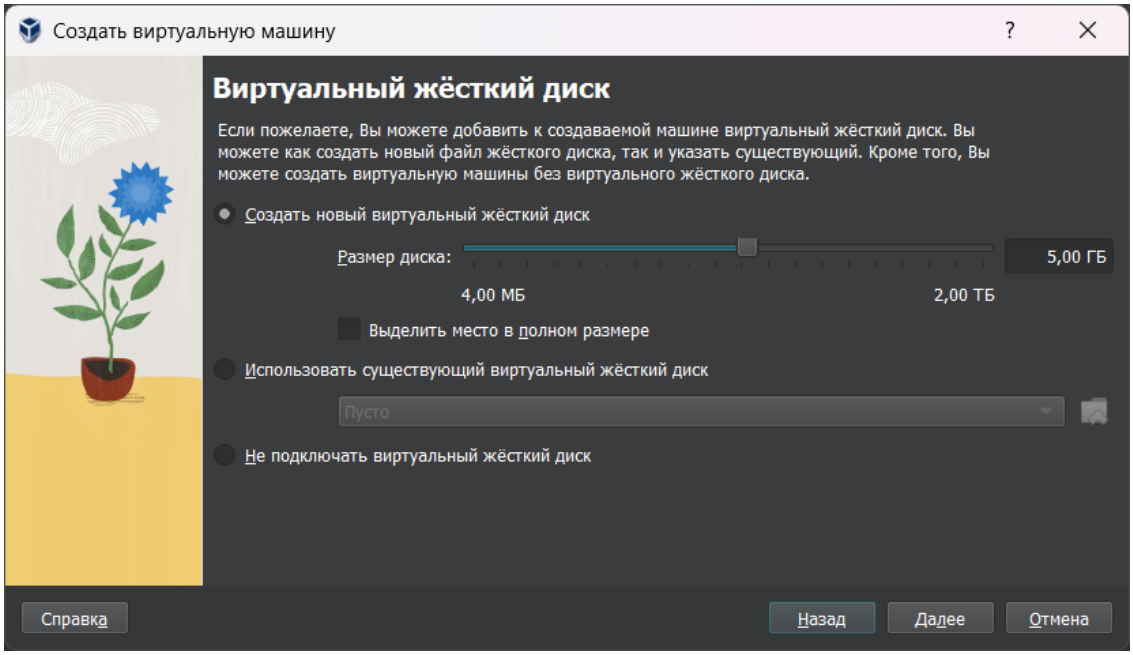
1. Задайте имя «master». В поле «Образ ISO» выберите путь до скачанного файла Ubuntu Server. Установите флаг «Пропустить автоматическую установку» и нажмите «Далее».



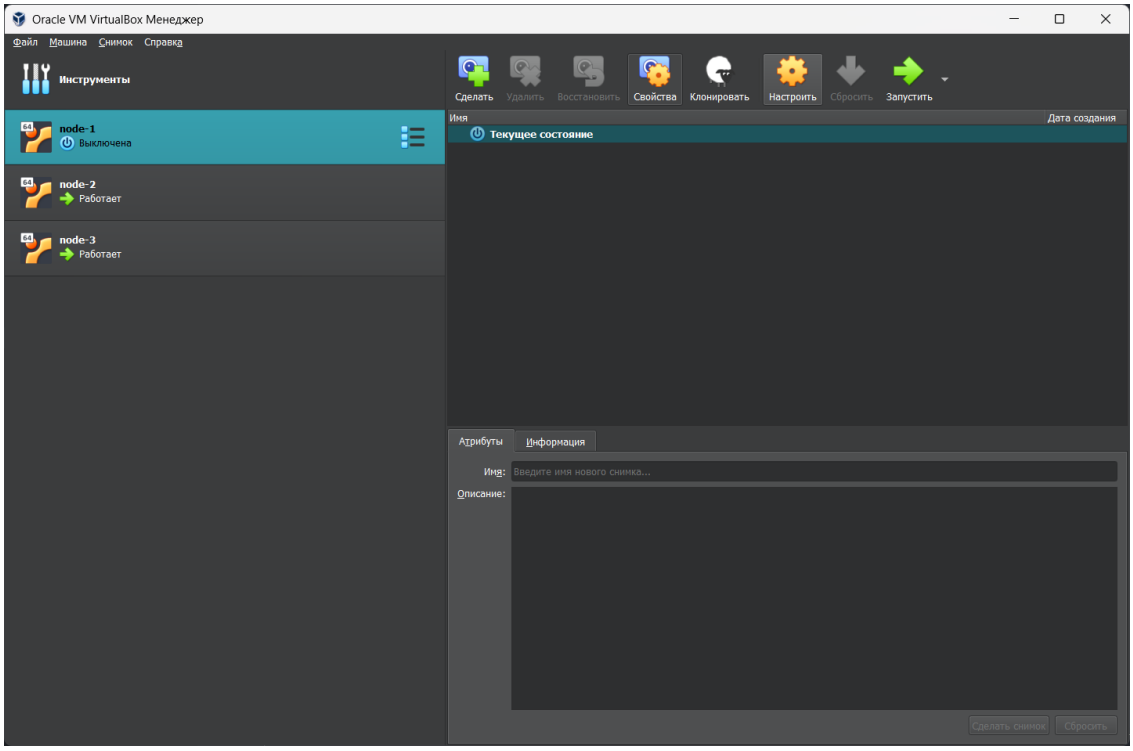
2. Установите параметры «Основная память» - 2048МБ и «Процессоры» - 1.



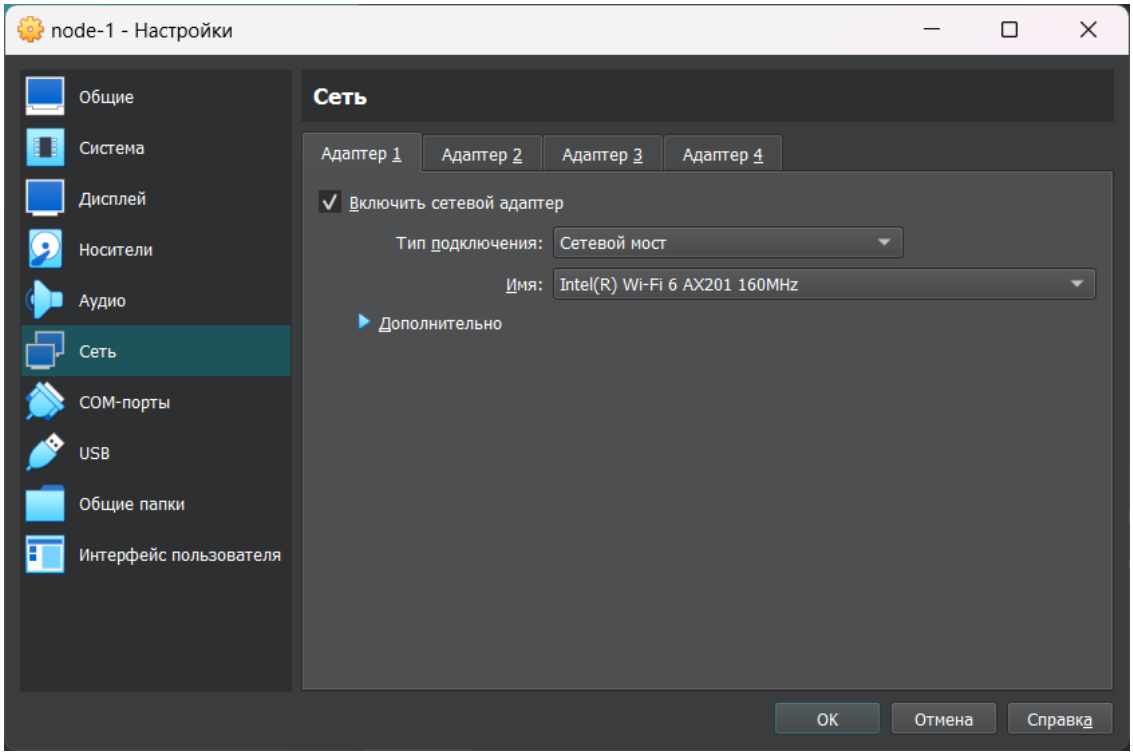
3. Выделите минимум 5ГБ пространства.



4. В главном меню выберите созданную ВМ и нажмите «Настроить».



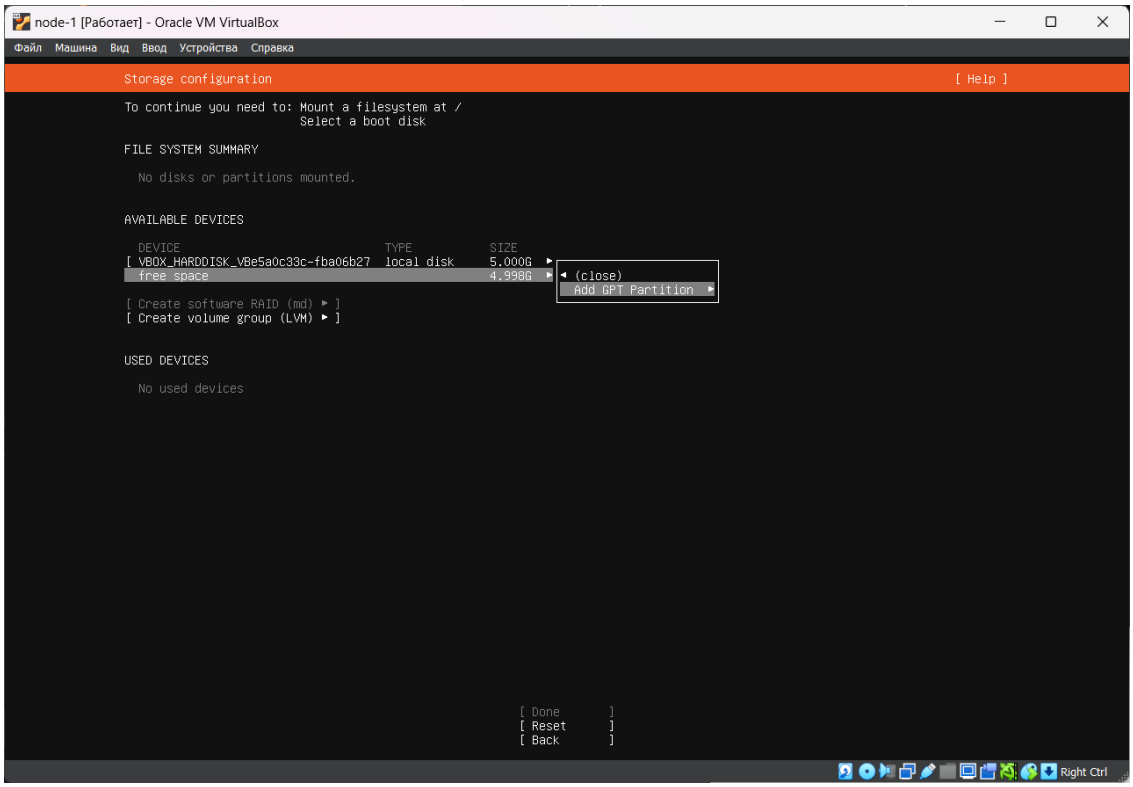
5. Перейдите в раздел «Сеть» → «Адаптер 1». Убедитесь, что стоит флаг в поле «Включить сетевой адаптер». В списке «Тип подключения» выберите «Сетевой мост».



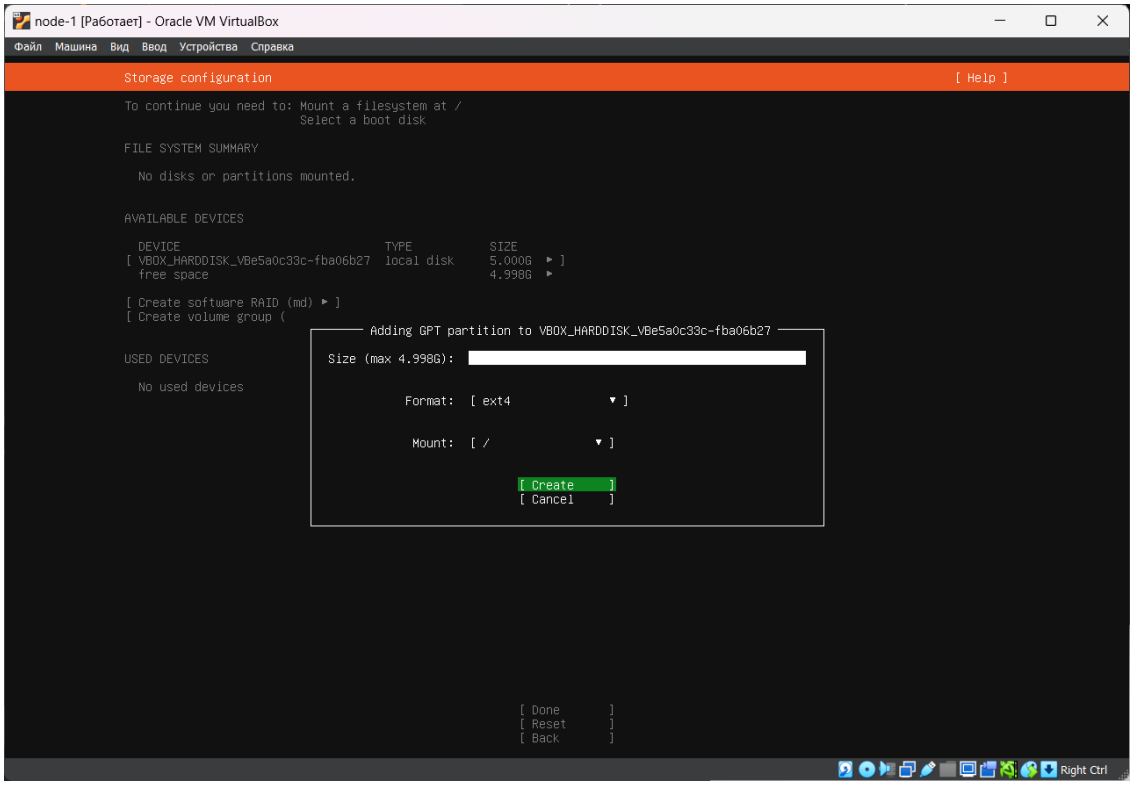
6. Запустите ВМ. После запуска выберите пункт «Try or Install Ubuntu Server».

7. Следующие 7 страниц настроек оставьте без изменений.

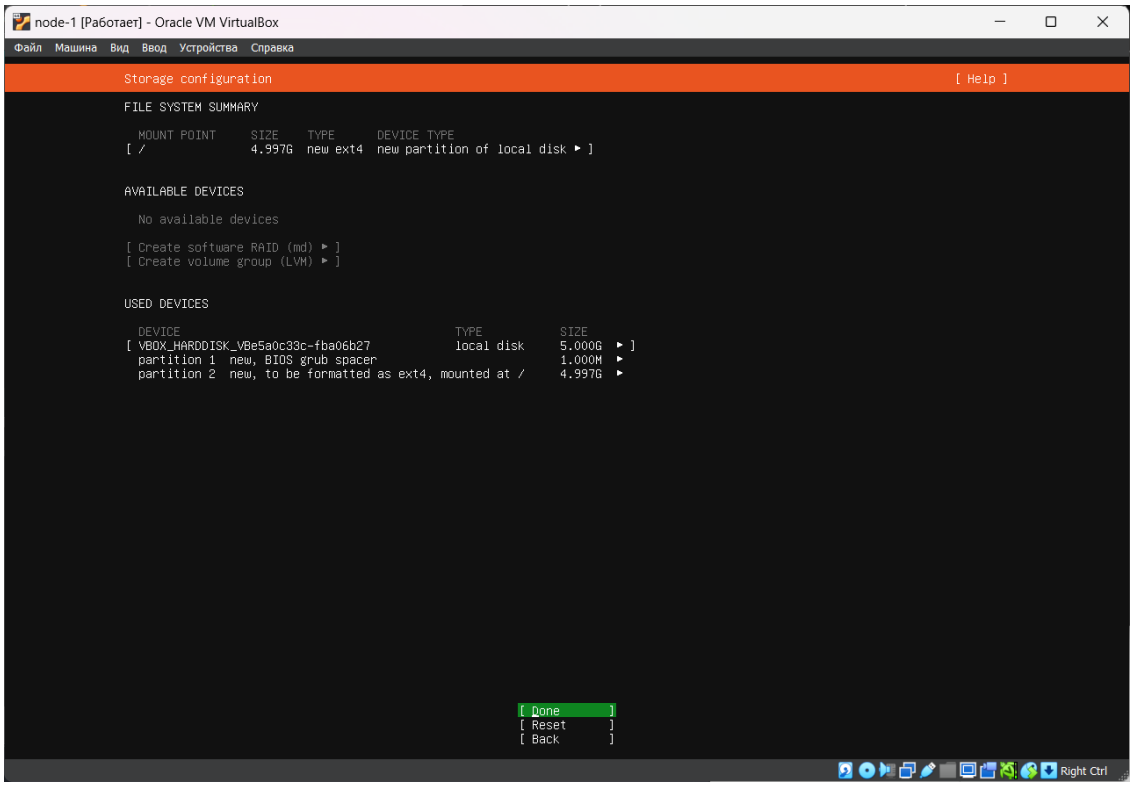
8. Перейдите на неразмеченную память и выберите «Add GPT Partition».



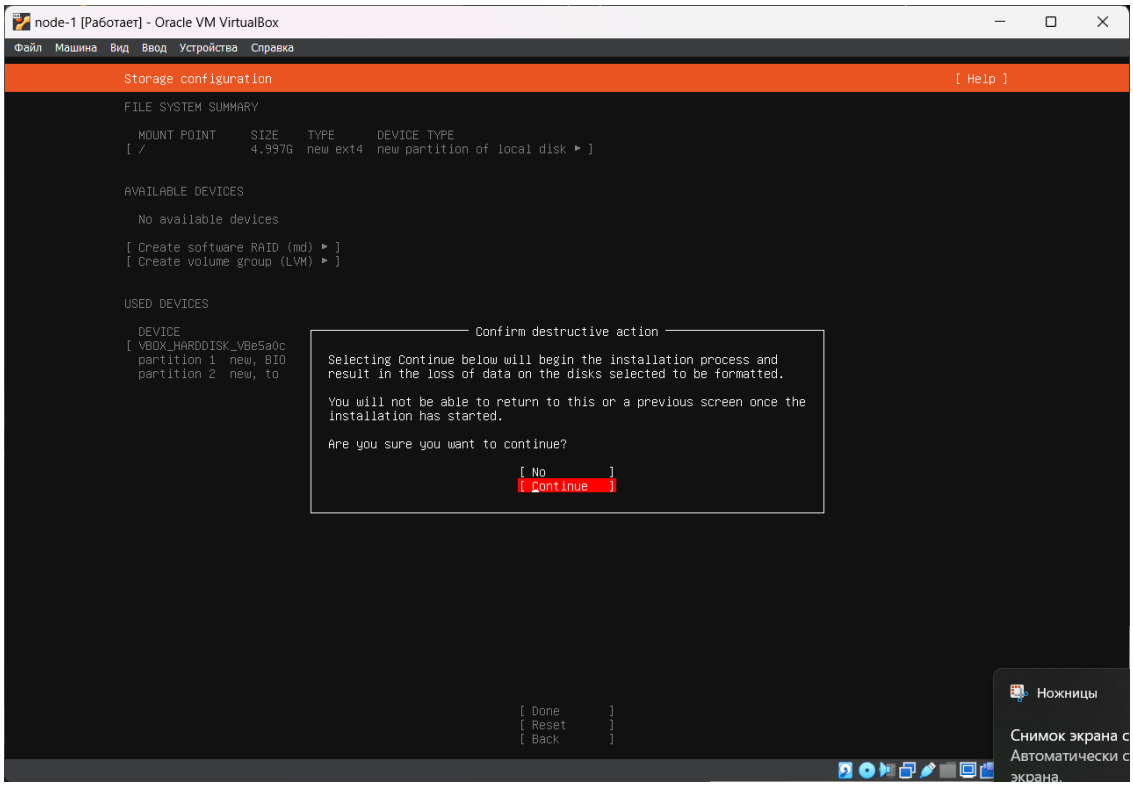
9. Оставьте поле размера пустым, чтобы выбрать всю область. В поле формата выберите «ext4», а в поле точки монтирования - «/».



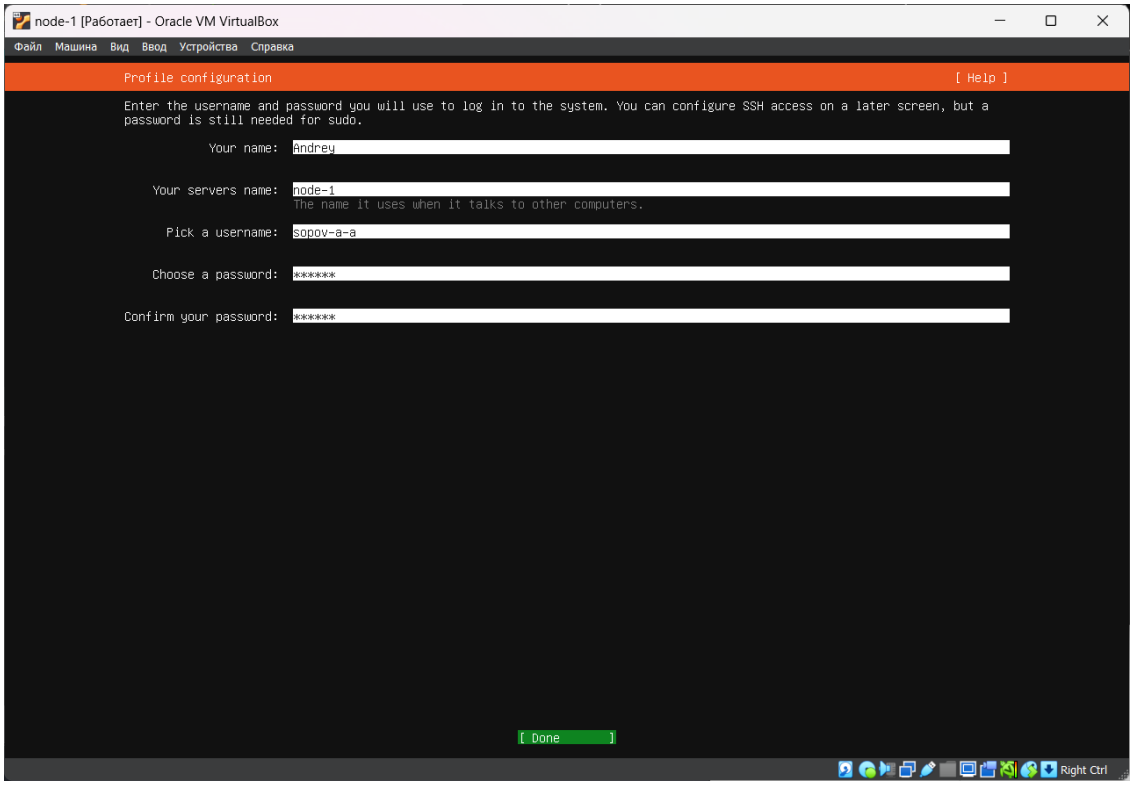
10. После вы должны увидеть примерно следующее:



Нажмите «Done», а затем в появившемся окне - «Continue».



11. Введите в поле «Your name» ваше имя, «Your servers name» - «node-1», «Pick a username» - ФИО в формате фамилия-и-о. Для дальнейшего удобства можете установить на всех ВМ один и тот же пароль.



12. Следующие 3 страницы настроек оставьте без изменений.

13. Дождитесь установки и перезагрузите ВМ.

14. Войдите в систему и установите ssh сервер: `sudo apt update && sudo apt install opensh-server -y`

15. Запустите демон ssh и проверьте, что служба работает: `sudo systemctl enable ssh && sudo systemctl status ssh.service`

Создайте ещё две виртуальные машины с Ubuntu Server с теми же параметрами, заменяя имена машин на «node-2» и «node-3» соответственно.

4.2 Создание инвентаря

Для начала создадим в домашней директории на хосте каталог `ansible`, где будут находиться все файлы, относящиеся к Ansible.

Предварительно узнайте IP-адреса виртуальных машин и создайте файл `hosts`, в котором опишите группы:

- `staging_serves`, состоящий из node-1 и node-2;

- `prod_serves`, состоящий из node-3;
- `all_serves`, объединяющий две предыдущие группы.

Также создайте файл `ansible.cfg`, в котором пропишите:

```
[defaults]
host_key_checking=false
```

4.3 Создание переменных группы

ПРИМЕЧАНИЕ: Лучшим способом подключения к серверам является аутентификация на основе SSH-ключей, но мы для простоты будем использовать пароль. В настоящих проектах никогда так не делайте.

Создайте файл для переменных группы `all_serves`, в котором определите две переменные:

```
ansible_user: <vm_username>
ansible_passowrd: <vm_password>
```

4.4 Проверка доступности хостов

Так как мы используем пароли при подключении, нам понадобится пакет `sshpass`. Установите его с помощью пакетного менеджера.

Убедитесь, что ваши ВМ запущены и проверьте их доступность с помощью ad-hoc команды `ping`. Если вы всё сделали верно, то увидите в консоли три успешно завершённые задачи.

4.5 Создание плейбука

Создайте роль `nginx`, состоящую из:

- задачи для установки сервера nginx последней версии с помощью модуля `apt`, которая будет вызывать обработчик запуска;
- обработчика, который будет запускать сервис nginx с помощью модуля `systemd`.

Создайте плейбук `nginx_install_and_run.yml`, который устанавливает nginx на все хосты, используя роль.

Для установки пакета необходимы root-права. Вставьте следующие инструкции в файл плейбука:

```
become:
  true
become_method:
  sudo
become_user:
  root
```

Ключевое слово `become` используется для повышения привилегий. `become_method` указывает на способ, которым мы хотим выполнить повышение привилегий, а `become_user` на пользователя, чьи привилегии мы хотим получить.

4.6 Запуск плейбука и проверка выполнения

Для запуска сценария используйте команду:

```
$ ansible-playbook -i hosts nginx_install_and_run.yml -kK
```

Дождитесь результатов работы плейбука. После успешной установки и запуска сервиса, при обращении по IP-адресам виртуальных машин вы увидите приветственную страницу nginx.

5 Контрольные вопросы

1. Что такое Ansible?

2. Что такое Playbook?

3. Для чего нужны роли?

4. Что гарантирует идемпотентность в Ansible?